Experiment 3

Student Name: Devansh Sharma **UID: 23BAI70166**
Branch: BE CSE-H AIML **Section/Group: 23AIT_KRG-1**
Semester: 6th **Date of Performance: 28/01/26**
Subject Name: Full Stack - II **Subject Code: 23CSH - 382**

1. **Title:**
   Redux Toolkit & Asynchronous State Management in EcoTrack.

2. **Aim:**
   To implement centralized state management in the EcoTrack application using Redux Toolkit and to handle asynchronous data operations using Redux async thunks with proper loading and error states.

3. **Objective:**
   After completing this experiment and its follow-up tasks, the student will be able to:
   - Configure a Redux store in a React application using Redux Toolkit
   - Create and integrate Redux slices for managing application data
   - Implement asynchronous actions using Redux async thunks
   - Manage loading, success, and error states during asynchronous operations
   - Connect React components to Redux state using React-Redux hooks
   - Trigger asynchronous data fetching through Redux actions from UI components
   - Use Redux state to derive filtered views without modifying the global store
   - Enhance user experience by handling refresh actions and improving async UI feedback.

4. Implementation/Code:

- **logSlice.jsx:**

```jsx
import {createSlice, createAsyncThunk} from "@reduxjs/toolkit";
import logs from '../data/logs';

export const fetchLogs = createAsyncThunk(
    "logs/fetchLogs",
    async () => {
        await new Promise((resolve) => setTimeout(resolve, 1000));
        return logs;
    }
)

const logSlice = createSlice({
    name: "logs",
    initialState: {
        data: [],
        status: "idle",
        error: null,
    },
    reducers: {},
    extraReducers:(builder) => {
        builder
        .addCase(fetchLogs.pending, (state) => {
            state.status = "loading";
        } ).addCase(fetchLogs.fulfilled, (state, action) => {
            state.status = "success";
            state.data = action.payload;
        }).addCase(fetchLogs.rejected, (state, action) => {
            state.status = "failed";
            state.error = action.error.message;
        })
    }
})
export default logSlice.reducer;
```

- **Store.jsx:**

```jsx
import { configureStore } from '@reduxjs/toolkit';
import logsReducer from "./logSlice"

const store = configureStore({
    reducer: {
        logs: logsReducer,
    },
});
export default store;
```

- **Logs.jsx:**

```jsx
import { useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";
import { fetchLogs } from "../store/logSlice";

const Logs = () => {
  const dispatch = useDispatch();
  const { data: logs, status, error } = useSelector((state) => state.logs);

  const handleRefresh = () => {
    dispatch(fetchLogs());
  }

  useEffect(() => {
    if (status === "idle") {
      dispatch(fetchLogs());
    }
  }, [status, dispatch]);

  if (status === "loading") {
    return <div>Loading logs...</div>;
  }

  if (status === "failed") {
    return <div>Error: {error}</div>;
  }

  const filteredArray = logs.filter((log) => log.carbon >= 4);
  const lowCarbonArray = logs.filter((log) => log.carbon < 4);
```

```jsx
  return (
    <div>
      <h2>High Carbon Activity</h2>
      {filteredArray.map((log) => (
        <div style={{ color: "red" }} key={log.id}>
          {log.activity} = {log.carbon} Kg
        </div>
      ))}
      <h2>Low Carbon Activity</h2>
      {lowCarbonArray.map((log) => (
        <div style={{ color: "green" }} key={log.id}>
          {log.activity} = {log.carbon} Kg
        </div>
      ))}
      <button onClick={handleRefresh}>Refresh Logs</button>
    </div>
  );
};

export default Logs;
```

- **AuthContext.jsx:**

```jsx
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export const AuthProvider = ({children}) => {
    const [isAuthenticated, setIsAuthenticated] = useState(false);

    return (
        <AuthContext.Provider value = {{isAuthenticated, setIsAuthenticated}}>
            {children}
        </AuthContext.Provider>
    )
}

export const useAuth = () => useContext(AuthContext);
```

- **ProtectedRoute.jsx**

```jsx
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import { children } from "react";

const ProtectedRoute = ({children}) => {
    const {isAuthenticated} = useAuth();

    if(!isAuthenticated) {
        return <Navigate to = "/login" replace/>
    }
    return children;
}

export default ProtectedRoute;
```

- **Login.jsx:**

```jsx
import { useAuth } from "../context/AuthContext";
import { useNavigate } from "react-router-dom";

const Login = () => {
    const {setIsAuthenticated} = useAuth();
    const navigate = useNavigate();

    const handleLogin = () => {
        setIsAuthenticated(true);
        navigate("/");
    }
    return (
        <>
        <h3>Login</h3>
        <button onClick={handleLogin}>Login</button>
        </>
    )
}
export default Login;
```

- **DashboardSummary.jsx:**

```jsx
import { useEffect, useState } from "react";
import { useSelector, useDispatch } from "react-redux";
import { fetchLogs } from "../store/logSlice";

const DashboardSummary = () => {
  const dispatch = useDispatch();
  const { data: logs, status, error } = useSelector((state) => state.logs);

  useEffect(() => {
    if (status === "idle") {
      dispatch(fetchLogs());
    }
  }, [status, dispatch]);

  const handleRefresh = () => {
    dispatch(fetchLogs());
  };

  if (status === "loading") {
    return <div>Loading summary...</div>;
  }

  if (status === "failed") {
    return <div>Error loading data: {error}</div>;
  }

  const total = logs.reduce((acc, log) => acc + log.carbon, 0);
```

```jsx
  return (
    <div>
      <h2>Carbon Footprint Summary</h2>
      <p>Total Carbon Footprint: {total} kg</p>
      <p>Total Activities: {logs.length}</p>
      <button onClick={handleRefresh}>Refresh Data</button>
    </div>
  );
};

export default DashboardSummary;
```

- **DashboardAnalytics.jsx**

```jsx
import { useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";
import { fetchLogs } from "../store/logSlice";

const DashboardAnalytics = () => {
  const dispatch = useDispatch();
  const { data: logs, status, error } = useSelector((state) => state.logs);

  useEffect(() => {
    const loadData = async () => {
      if (status === "idle") {
        dispatch(fetchLogs());
      } else {
        await new Promise(resolve => setTimeout(resolve, 1000));
      }
    };
    loadData();
  }, [status, dispatch]);

  const handleRefresh = () => {
    dispatch(fetchLogs());
  };

  if (status === "loading") {
    return <div>Loading analytics...</div>;
  }

  if (status === "failed") {
    return <div>Error loading analytics: {error}</div>;
  }

  const highCarbonCount = logs.filter(log => log.carbon >= 4).length;
  const lowCarbonCount = logs.filter(log => log.carbon < 4).length;
  return (
    <div>
      <h2>Carbon Analytics</h2>
      <p>High Carbon Activities: {highCarbonCount}</p>
      <p>Low Carbon Activities: {lowCarbonCount}</p>
      <p>Average Carbon per Activity: {averageCarbon} kg</p>
      <button onClick={handleRefresh}>Refresh Analytics</button>
    </div>
  );
};

export default DashboardAnalytics;
```

- **DashboardLayout.jsx**

```jsx
import { Link, Outlet } from "react-router-dom"

const DashboardLayout = () => {
    return (
        <>
            <h3>Dashboard</h3>
            <nav style={{display: 'flex', gap: '20px'}}>
                <Link to="summary">Summary</Link>
                <Link to="analytics">Analytics</Link>
                <Link to ="logs">Logs</Link>
            </nav>
            <hr />
            <Outlet />
        </>
    )
}

export default DashboardLayout;
```

- **Dashboard**

```jsx
const Dashboard = ({ carbonLogs }) => {
  const total = carbonLogs.reduce((acc, x) => {
    return acc + x.carbon;
  }, 0);

  return (
    <div>
      <p>Total Carbon Footprint: {total} kg</p>
      <ul>
        {carbonLogs.map((log) => (
          <li key={log.id}>
            {log.activity} = {log.carbon} kg
          </li>
        ))}
      </ul>
    </div>
  );
};

export default Dashboard;
```

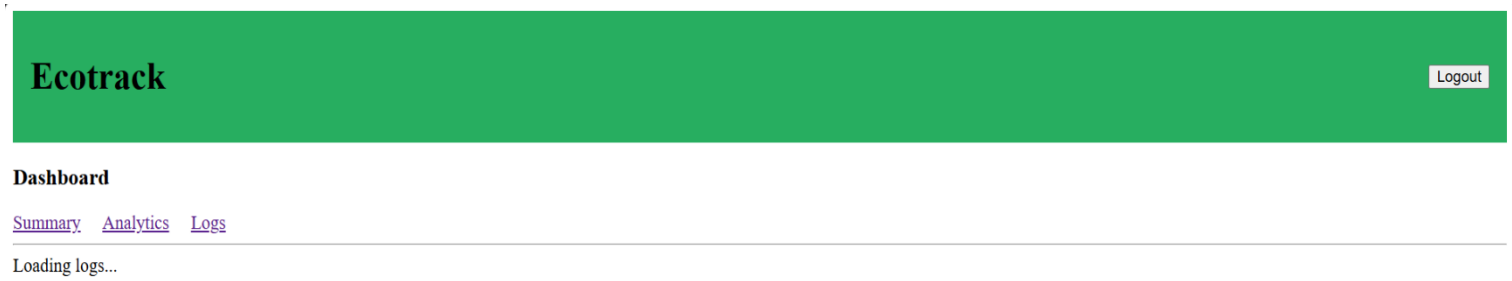
- **App.jsx**

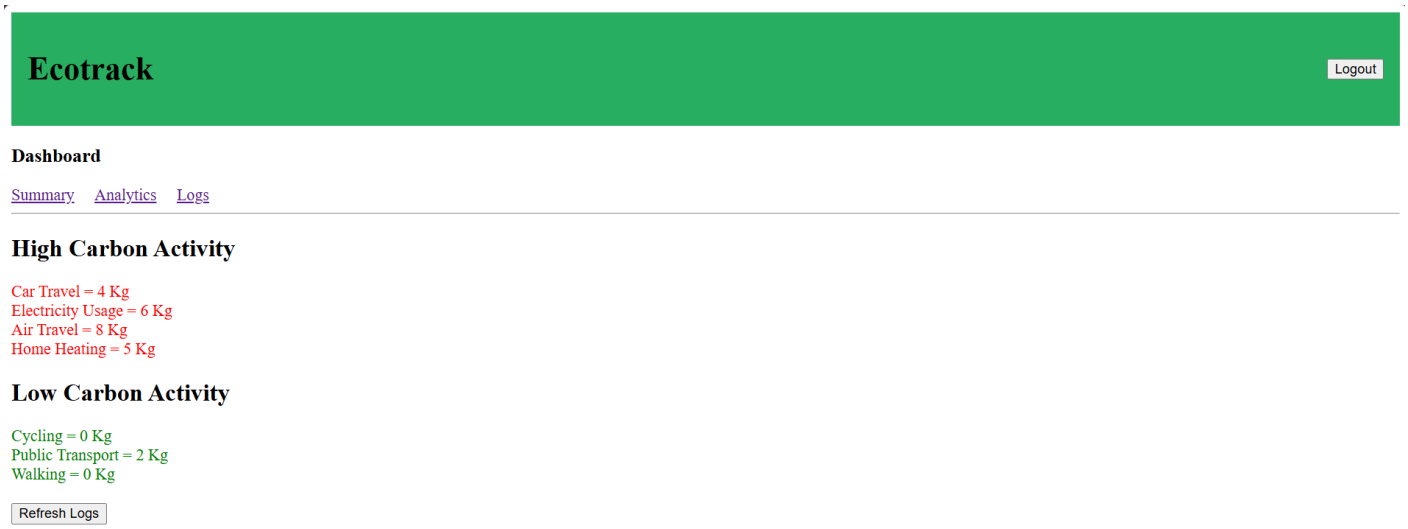
```jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import DashboardAnalytics from "./pages/DashboardAnalytics";
import Logs from "./pages/Logs";
import Login from "./pages/login";
import DashboardLayout from "./pages/DashboardLayout";
import DashboardSummary from "./pages/DashboardSummary";
import ProtectedRoute from "./routes/ProtectedRoute.jsx";
import Header from "./components/Header";

function App() {
  return (
    <BrowserRouter>
      <Header title="EcoTrack" />
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/" element={<ProtectedRoute><DashboardLayout /></ProtectedRoute>}>
          <Route index element={<DashboardSummary />} />
          <Route path="summary" element={<DashboardSummary />} />
          <Route path="analytics" element={<DashboardAnalytics />} />
          <Route path="logs" element={<Logs />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

5. Output:





6. Learning Outcomes:

- We learnt about React Apps and how to create them.
- We learnt about redux and it’s components.
- We learnt about the use of thunks andSlices.
- We learnt about Authentication and index pages
- We learnt the use of useContext and useState.
- We learnt about the flow of a React project.